Security Design

Core principles

- Financial correctness, least privilege and credential safety take priority.
- Normal HTTP traffic passes through YARP Gateway.
- Gateway authentication does not replace service/domain authorization.
- MSSQL is the durable authentication/financial source of truth.
- Redis is transient support state.
- Production secrets are not committed and startup fails fast when required secrets are absent.
- Passwords, OTPs, JWTs, refresh tokens, service keys and connection secrets are never logged.
- Ownership/risk facts are server-derived.

Gateway trust model

Client -> Gateway: protected /api/* routes require JWT; register/verify/login/refresh are anonymous.

Gateway -> FinWallet.Api: Gateway supplies a downstream service key; the API independently revalidates JWT and ownership.

FinWallet.Api -> Gateway /providers/*: requires InternalServiceKey.

Gateway -> Fake provider: requires a separate DownstreamServiceKey.

Health routes remain open for orchestrator/YARP checks; Swagger exposure is environment-configured.

HTTP attack surface

FinWallet.Shared.Web applies:

- Kestrel server header disabled;
- TRACE/CONNECT blocked;
- JSON required for body-bearing POST/PUT/PATCH;
- body/header-count/header-size limits;
- request-header/keep-alive timeouts;
- max concurrent connections;
- per-IP fixed-window rate limit;
- zero queue by default;
- CORS allow-list;
- bounded correlation ID;
- no-store/no-cache;
- CSP plus frame/MIME/referrer/permissions/cross-origin headers.

These controls reduce L7 abuse/resource exhaustion; volumetric DDoS still requires cloud/edge/ingress/WAF protections.

Passwords

V1:

- PBKDF2-HMAC-SHA512;

- 220,000 iterations;
- 32-byte random salt;
- 64-byte hash;
- constant-time comparison;
- persisted hash version.

Iteration count is not a loose runtime tuning value; future change requires versioned migration/rehash.

JWT and sessions

JWT uses fixed HMAC-SHA256 with minimal sub, sid, JTI and iat claims. It contains no balance/IBAN/contact data. Issuer/audience/signing key come from configuration/secret store. Lifetime is configurable only within a safe range. Signing algorithm is not configurable.

High-risk transfer validates sid against MSSQL CustomerSession state so a revoked session can stop money movement before its short-lived JWT expires.

Refresh tokens

Clients receive opaque random tokens; raw tokens are not persisted. Rotation is single-use through compare-and-set semantics. Reuse of a consumed token revokes the session/token family.

OTP

Redis stores an HMAC digest rather than raw OTP. Lua scripts provide atomic issue/attempt/consume behavior. Verification fails closed when Redis is unavailable.

SQL injection

Financial/auth persistence uses parameterized SqlClient commands. Request text is never used as a dynamic SQL fragment.

BOLA / ownership

- customer identity comes from JWT sub;
- owned-resource queries include customer boundaries;
- transfer source ownership is validated before fraud and again inside locked posting;
- destination/currency/lifecycle are revalidated before commit.

Fraud security

Clients cannot supply trust flags such as isNewDevice, known beneficiary, velocity or 24-hour amount. Signals come from durable server state.

```
completed replay
-> server-side risk signals
-> internal fraud
-> external fraud
-> combined decision
-> posting only on Allow
```

External fraud unavailable/malformed => fail closed.

Idempotency / replay

Money-changing transfer requires Idempotency-Key. Durable identity is Scope + CustomerId + Key; a canonical request hash turns same-key/different-payload reuse into a conflict.

Logging

Never log:

- password/hash/salt;
- OTP/digest/pepper;
- JWT/refresh token;
- Authorization header;
- internal/downstream keys;
- signing key;
- SQL/Redis connection strings containing credentials;
- unmasked phone/email/IBAN/account values.

OWASP/API security summary

- Broken Access Control/BOLA: gateway + service auth and owner-aware SQL.
- Authentication failures: JWT/session/refresh rotation/login lockout/OTP/rate limit.
- Injection: parameterized SQL.
- Security Misconfiguration: prod Swagger off, bounded HTTP config, no server header.
- Cryptographic failures: PBKDF2, HMAC, short JWT, secret management.
- Sensitive business flows: durable idempotency, fraud and ledger.
- Resource consumption: rate/body/header/connection/timeouts.
- SSRF/unsafe API consumption: provider URLs are server-owned configuration.
- Supply chain: central package versions and CI build/test; SBOM/vulnerability scanning remains open work.

Remaining security work

- Gateway bypass/rate-limit integration tests;
- dependency vulnerability/SBOM scanning;
- centralized masked logging/SIEM/alerting;
- NetworkPolicy/ingress/TLS hardening;
- logout/session-revoke endpoint;
- durable FraudEvents/manual review;
- incident/reconciliation runbooks.